AdaBoost vs Gradient Boosting: A Comparative Study of Boosting Algorithms for Classification

Student ID – 24106862 Student Name - Lojana Jegatheeswaran

Introduction

In supervised machine learning, improving predictive performance is a key objective, particularly when individual models fail to achieve high accuracy. Ensemble learning techniques address this limitation by combining multiple models to produce a more robust and reliable prediction system. Among these techniques, boosting algorithms have gained significant attention due to their ability to transform weak learners into strong classifiers.

This tutorial aims to compare AdaBoost and Gradient Boosting to understand how boosting techniques improve classification performance. By conducting experiments on a real-world dataset, this study evaluates their effectiveness and highlights the differences between these two widely used ensemble methods.

Boosting Concept

Boosting is an ensemble learning technique that aims to improve the performance of weak learners by combining them into a strong predictive model. A weak learner is defined as a model that performs only slightly better than random guessing. The key idea behind boosting is to train models sequentially, where each new model focuses on correcting the errors made by previous ones.

Boosting has been shown to significantly improve the accuracy of learning algorithms by reducing both bias and variance. According to (Freund and Schapire, 1999), boosting provides a general framework for enhancing the performance of any given learning algorithm. This makes it a powerful technique in supervised machine learning, particularly for classification tasks.

AdaBoost Algorithm

AdaBoost (Adaptive Boosting) is one of the earliest and most influential boosting algorithms. It works by assigning weights to each training sample and updating these weights iteratively based on classification errors. Initially, all samples are given equal weights. During each iteration, a weak learner (typically a decision stump) is trained, and its performance is evaluated. Misclassified samples are assigned higher weights, while correctly classified samples receive lower weights. This process forces subsequent learners to focus on difficult samples. The final model is constructed as a weighted combination of all weak learners.

(Freund and Schapire, 1997) demonstrated that AdaBoost can significantly reduce classification error by focusing on previously misclassified instances. However, the algorithm may be sensitive to noisy data and outliers due to its emphasis on difficult samples.

The working process of AdaBoost can be summarized as follows:

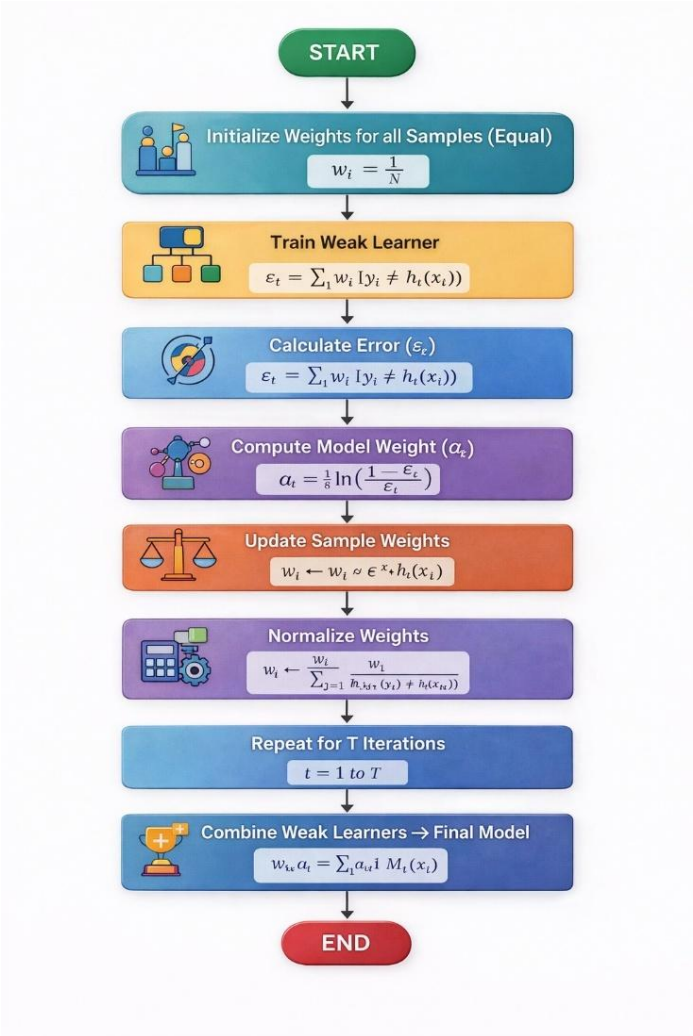

Figure 1: Algorithmic Workflow of AdaBoost

AdaBoost combines multiple weak learners to form a strong classifier by focusing more on misclassified samples.

Initially, all training samples are assigned equal weights:

$$w_i = \frac{1}{N}, \quad i = 1,2,\dots,N$$

At each iteration t, a weak learner $h_t(x)$ is trained, and its error is calculated as:

$$\epsilon_t = \sum_{i=1}^{N} w_i \cdot I\big(y_i \neq h_t(x_i)\big)$$

Where:

ϵ_t is the weighted error,

$I\big(y_i \neq h_t(x_i)\big) = 1$ if the sample is misclassified, and 0 otherwise.

The importance of the weak learner is computed as:

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

The sample weights are then updated:

$$w_i(t+1) = w_i(t) \cdot e^{-\alpha_t y_i h_t(x_i)}$$

Where:

$y_i \in \{-1,+1\}$ is the true label.

$h_t(x_i) \in \{-1,+1\}$ is the predicted label.

After updating the weights are normalized so that they sum to 1.

The final prediction is obtained by combining all weak learners:

$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

Gradient Boosting Algorithm

Gradient Boosting is a more advanced boosting technique that generalizes the boosting framework by formulating it as an optimization problem. Instead of reweighting samples like AdaBoost, Gradient Boosting builds models by minimizing a loss function using gradient descent.

As introduced by (Friedman, 2001), Gradient Boosting constructs the model in a stage-wise manner by fitting each new model to the residual errors of the previous model. These residuals represent the difference between the actual values and the predicted values.

At each iteration, the algorithm computes the negative gradient of the loss function, which serves as the target for the next model. This allows the algorithm to progressively reduce prediction errors and improve performance.

Gradient Boosting is more flexible than AdaBoost because it can optimize different types of loss functions, making it suitable for both classification and regression tasks. Additionally, it is generally more robust to noise compared to AdaBoost.

The working process of Gradient Boosting can be summarized as follows:

Figure 2: Algorithmic Workflow of Gradient Boosting

The Gradient Boosting model is defined as an additive model:

$$F(x) = \sum_{m=1}^{M} \gamma_m h_m(x)$$

Where:

$h_m(x)$ represents the weak learner at iteration m,

γ_m is the learning rate (step size) controlling how much each weak learner contributes to the final model.

The model is updated iteratively as:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

At each iteration, the algorithm computes the residuals (negative gradients):

$$r_{im} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

Where:

r_{im} is the residual for sample i at iteration m,

L is the loss function,

y_i is the true value,

$F(x_i)$ is the predicted value.

Key Differences Between AdaBoost and Gradient Boosting

Aspect	AdaBoost	Gradient Boost
Learning strategy	Reweights training samples	Fits residual errors
Optimization method	Indirect	Direct loss minimization
Sensitivity to noise	High	Lower
Complexity	Simple	More advanced
Flexibility	Limited	High

AdaBoost focuses on adjusting the importance of training samples, while Gradient Boosting focuses on reducing prediction errors through gradient-based optimization. As a result, Gradient Boosting often achieves better performance on complex datasets.

Dataset Description

The experiments in this study were conducted using the Digits dataset, which is a widely used benchmark dataset for multi-class classification tasks. The dataset is available through the Scikit-learn library and consists of images of handwritten digits ranging from 0 to 9.

The dataset contains 1797 samples, where each sample represents an image of a handwritten digit. Each image is of size 8 × 8 pixels, resulting in 64 numerical features per sample. These features correspond to grayscale intensity values of each pixel in the image.

The target variable consists of 10 classes, representing the digits:

- 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9

The dataset is relatively well-balanced across classes and does not contain missing values, making it suitable for evaluating classification algorithms. Due to its multi-class nature and moderate complexity, it provides a more challenging problem compared to binary classification datasets and allows a more meaningful comparison of boosting techniques.

Data Preparation

The Digits dataset was loaded using the Scikit-learn library. To evaluate model performance, the dataset was divided into training and testing sets using an 80:20 split. A stratified sampling approach was applied to ensure that all digit classes were proportionally represented in both subsets.

No additional preprocessing steps such as handling missing values were required, as the dataset is complete. Feature scaling was not applied because tree-based models, including AdaBoost and Gradient Boosting, are generally insensitive to the scale of input features.

Models Used

To compare the effectiveness of boosting techniques, three models were implemented and compared:

1. **Weak Learner (Decision Stump)**

A decision tree classifier with a maximum depth of one was used as the weak learner. This model serves as a baseline for comparison, as it represents a simple classifier with limited ability to capture complex patterns in the data.

2. **AdaBoost Model**

The AdaBoost algorithm was implemented using decision stumps as base learners. The model was trained iteratively, where each weak learner

focused more on samples that were misclassified in previous iterations. A fixed number of estimators was used to construct the final ensemble model, and the predictions were combined using a weighted voting mechanism.

3. Gradient Boosting Model

The Gradient Boosting algorithm was implemented to build an additive model in a stage-wise manner. Each new model was trained to predict the residual errors of the previous model, allowing the overall model to progressively minimize the loss function. Compared to AdaBoost, this approach provides greater flexibility and the ability to model more complex relationships.

Evaluation Metrics

Since the Digits dataset is a multi-class classification problem, model performance was evaluated using metrics that consider all classes simultaneously.

- Accuracy: Measures the overall proportion of correctly classified instances across all classes.
- Precision (weighted): Measures the proportion of correctly predicted positive instances among all predicted positive instances, weighted by class frequency.
- Recall (weighted): Measures the proportion of correctly identified instances among all actual instances, weighted by class frequency.
- F1-score (weighted): Harmonic mean of precision and recall, providing a balanced measure of performance.

Weighted averaging was used for precision, recall, and F1-score to account for class imbalance and ensure that all classes contribute proportionally to the final evaluation.

In addition, confusion matrices were used to provide a detailed visualization of classification performance across all digit classes, enabling analysis of misclassification patterns.

Results and Discussion

1. Comparison of Model Performance Metrics

The weak learner (decision stump) achieved very low performance, with an accuracy of 0.1944. This indicates that the model is unable to effectively learn patterns from the Digits dataset due to its simplicity. The extremely low precision (0.0705) and F1-score (0.0936) further confirm that the weak learner performs poorly in this multi-class classification task.

AdaBoost shows a significant improvement over the weak learner, achieving an accuracy of 0.8306. The precision (0.8367), recall (0.8306), and F1-score (0.8291) indicate balanced and consistent performance across different classes. This demonstrates the effectiveness of AdaBoost in combining multiple weak learners to improve classification accuracy.

Gradient Boosting achieves the highest performance among all models, with an accuracy of 0.9139. The precision (0.9136), recall (0.9139), and F1-score (0.9132) are all consistently high, indicating strong and reliable classification performance. This suggests that Gradient Boosting is more effective in capturing complex patterns in the Digits dataset.

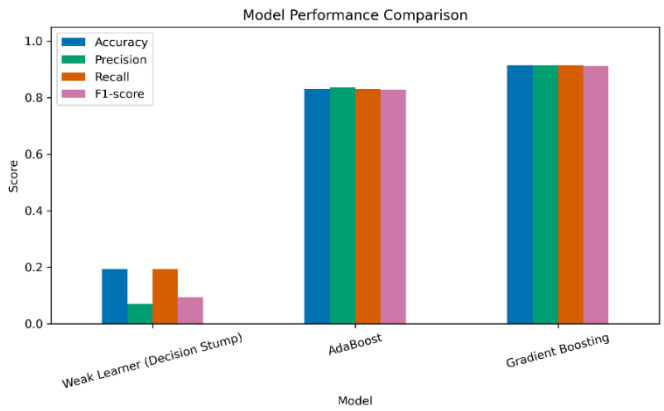

Figure 3: Comparison of model performance across accuracy, precision, recall, and F1-score.

2. Training and Testing Accuracy Analysis

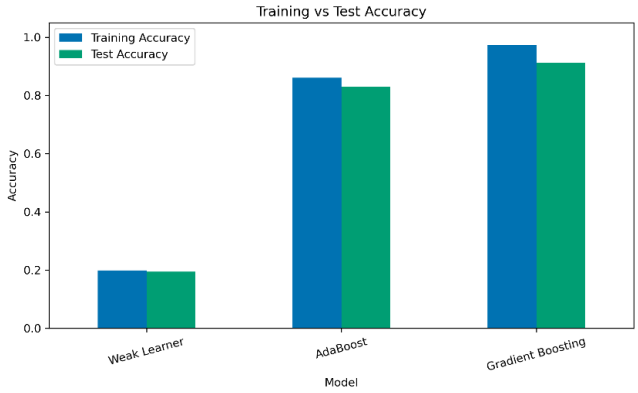

Figure 4: Comparison of training and testing accuracy for all models.

Figure 4 illustrates the relationship between training and testing accuracy for each model, providing insight into model generalization.

The weak learner shows low accuracy on both training (~0.20) and testing (~0.19) datasets, indicating underfitting, as the model is too simple to capture meaningful patterns in the data.

AdaBoost achieves higher training accuracy (~0.86) and test accuracy (~0.83), with a small gap between them. This indicates that the model generalizes well and does not significantly overfit the training data.

Gradient Boosting achieves the highest training accuracy (~0.97) and test accuracy (~0.91). Although there is a slightly larger gap between training and testing performance compared to AdaBoost, this suggests a mild level of overfitting. However, the model still maintains strong generalization and delivers the best overall performance.

3. Effect of Number of Estimators

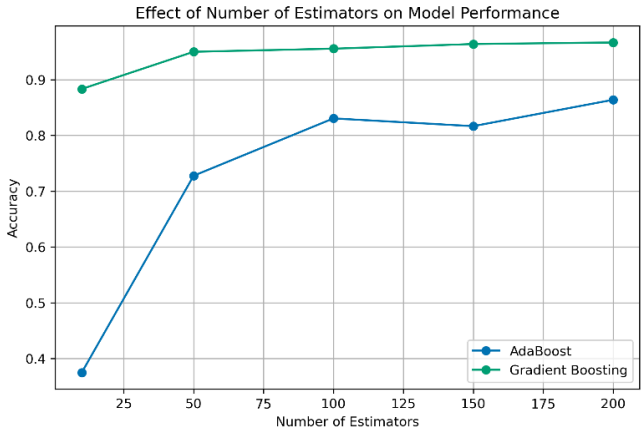

Figure 5: Effect of the number of estimators on the performance of AdaBoost and Gradient Boosting.

To further analyse model performance, the number of estimators was varied for both AdaBoost and Gradient Boosting. As shown in Figure 5, AdaBoost starts with relatively low accuracy when the number of estimators is small but improves significantly as more weak learners are added. However, its performance fluctuates slightly and does not reach the same level as Gradient Boosting. In contrast, Gradient Boosting achieves high accuracy even with a small number of estimators and shows steady improvement before stabilising. This indicates that Gradient Boosting is more effective at capturing complex patterns in the data, while AdaBoost requires more estimators to achieve comparable performance.

Error Analysis: Confusion Matrix and Misclassified Samples

To further evaluate model performance, confusion matrices and misclassified samples were analysed for all models.

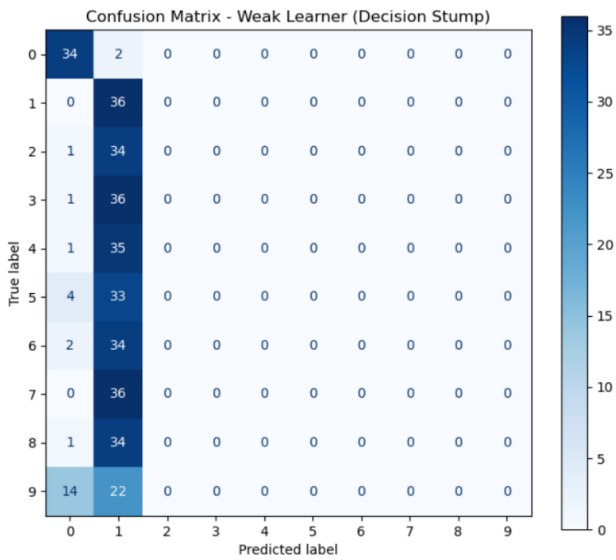

Figure 6: Confusion matrix showing poor performance and underfitting.

The confusion matrix for the weak learner (decision stump) shows that most predictions are concentrated in a few classes, indicating severe underfitting and poor classification ability. This confirms that a single weak learner is not sufficient to capture the complexity of the dataset.

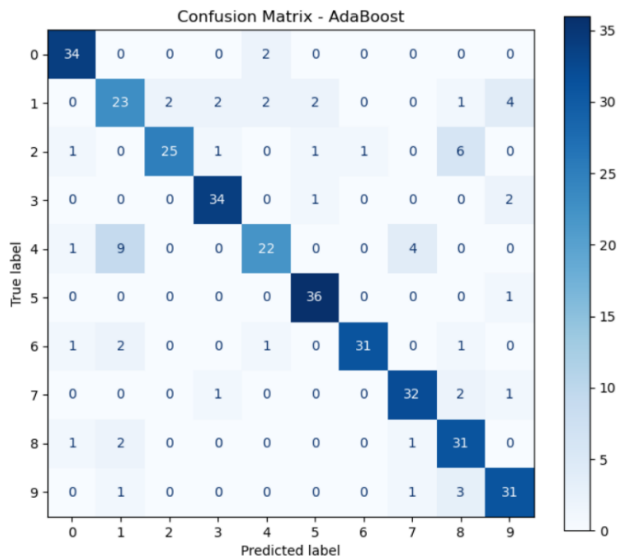

Figure 7: Confusion matrix showing improved classification with some errors.

In contrast, the AdaBoost confusion matrix shows a clear improvement, with more predictions correctly classified along the diagonal. However, several off-diagonal values remain, indicating misclassifications between certain digits. These errors suggest that while AdaBoost improves performance, it still struggles to fully distinguish between visually similar classes.

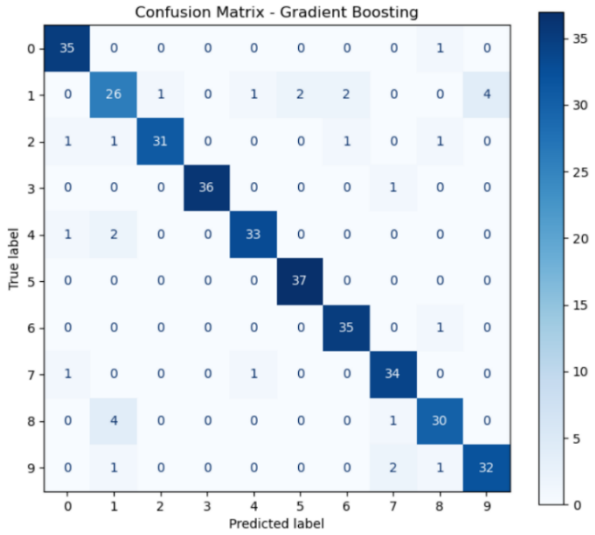

Figure 8: Confusion matrix showing high accuracy with minimal errors.

The Gradient Boosting confusion matrix demonstrates the best performance, with most values concentrated along the diagonal and very few misclassifications. This indicates that the model can accurately classify most samples and capture complex patterns in the data.

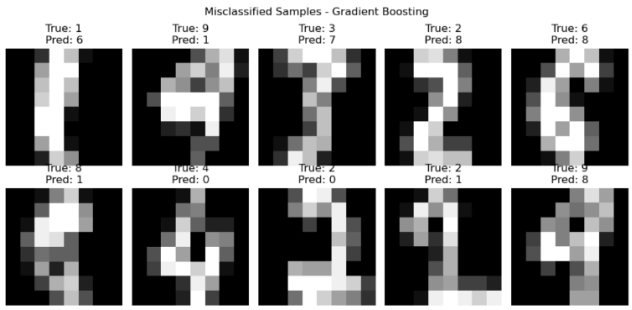

Figure 9: Misclassified digit samples highlighting differences between true and predicted labels.

To gain further insight, misclassified samples from the Gradient Boosting model were visualised. These examples show that errors typically occur when digits have similar shapes or unclear structures, such as confusion between digits like 1, 7, and 9. In some cases, the handwritten digits appear ambiguous or incomplete, making them difficult even for the model to classify correctly.

Overall, this analysis highlights that Gradient Boosting not only achieves higher numerical performance but also demonstrates more reliable and interpretable classification behaviour. Visualising misclassified samples provides valuable insight into model limitations and enhances understanding beyond standard evaluation metrics.

Discussion

While both AdaBoost and Gradient Boosting improve predictive performance through sequential learning, their underlying optimization mechanisms fundamentally differ. AdaBoost implicitly minimizes an exponential loss function by reweighting misclassified samples, which increases its sensitivity to noisy data and outliers. In contrast, Gradient Boosting explicitly formulates the learning process as a gradient descent optimization problem in function space, allowing it to minimize a wide range of differentiable loss functions.

This flexibility enables Gradient Boosting to better capture complex, non-linear relationships in the data, which is particularly evident in the Digits dataset where class boundaries are not linearly separable. Furthermore, Gradient Boosting's iterative fitting of residuals allows it to progressively refine errors in a more stable manner compared to AdaBoost's aggressive reweighting strategy.

However, this increased flexibility comes at the cost of higher computational complexity and a greater risk of overfitting if hyperparameters such as learning rate and number of estimators are not carefully tuned. In this study, although a slight gap between training and testing accuracy was observed for Gradient Boosting, the model maintained strong generalization performance, indicating that overfitting was effectively controlled.

Overall, the results suggest that while AdaBoost is effective for simpler problems, Gradient Boosting provides a more robust and scalable solution for complex classification tasks.

Conclusion

This study demonstrated that boosting algorithms significantly enhance classification performance compared to a single weak learner. While AdaBoost effectively improves accuracy by focusing on misclassified samples, Gradient Boosting provides a more flexible and powerful framework by directly minimizing a loss function through gradient descent. The experimental results show that Gradient Boosting achieves superior performance on the Digits dataset due to its ability to model complex patterns and refine residual errors iteratively. Therefore, Gradient Boosting is recommended for more complex classification tasks, whereas AdaBoost remains a simpler and computationally efficient alternative for less complex problems.

Code Availability

The code for this project is available at:

https://github.com/Lojana96/boosting-tutorial-digits-classification

Accessibility Considerations

To improve accessibility, colour-blind friendly visualisation practices were adopted by using a high-contrast colour palette (seaborn colour-blind style) along with clear labels and markers to ensure all figures are interpretable by a wider audience.

References

Freund, Y. and Schapire, R. (1999). A Short Introduction to Boosting. *Journal of Japanese Society for Artificial Intelligence*, [online] 14(5), pp.771–780. Available at: https://www.schapire.net/papers/FreundSc99.pdf [Accessed 27 Mar. 2026].

Freund, Y. and Schapire, R.E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1), pp.119–139. doi:https://doi.org/10.1006/jcss.1997.1504.

Friedman, J.H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), pp.1189–1232. doi:https://doi.org/10.1214/aos/1013203451.